

AI Harness Engineering: A Runtime Substrate for Foundation-Model Software Agents

Hailin Zhong¹, Shengxin Zhu^{2,*}

¹Hong Kong Baptist University, Hong Kong, China

²Beijin Normal University, Zhuhai, China

*Correspondence: shengxin.zhu@bnu.edu.cn

Abstract

Foundation models have transformed automated code generation, yet autonomous software-engineering agents remain unreliable in realistic development settings. The dominant explanation locates this gap in model capability. We propose a different locus: software-engineering capability emerges from a *model–harness–environment system*, in which a runtime substrate—the harness—mediates how a foundation-model agent observes a project, acts on it, receives feedback, and establishes that a change is complete. We formalize this substrate as an **AI Harness Engineering** and identify eleven component responsibilities: task specification, context selection, tool access, project memory, task state, observability, failure attribution, verification, permissions, entropy auditing, and intervention recording. We operationalize the harness through a four-level ladder (H0–H3) that progressively exposes runtime support to the agent, and we propose a trace-based evaluation protocol that converts each agent run into an auditable episode package. Applied to a controlled validation task, the framework yields episode packages whose evidence structure varies systematically with harness level: lower levels produce only a final patch, higher levels produce reproduction logs, failure attributions, deterministic requirement checks, and structured verification reports. The framework reframes the central question of autonomous software engineering from whether a foundation model can produce a patch to whether the model–harness–environment system can produce a *verifiably* correct, attributed, and maintainable change. We outline a research program for the runtime systems that foundation-model software agents will require.

Keywords: harness engineering; foundation models; autonomous software engineering; runtime systems; agent evaluation; verification; software-engineering agents.

Introduction

Foundation models have rapidly become capable programming assistants [1, 2]. They generate functions, modify files, explain code, write tests, invoke tools, and interact with software repositories. This progress has energized an ambition that predates the current generation of models but has only recently become plausible: autonomous software-engineering agents that take a high-level development task and carry it through implementation, testing, verification, and maintenance with minimal human supervision.

Recent benchmarks and agent systems [3–7] show that the gap between local code generation and complete software work is real and persistent. A model that writes a correct local patch may still fail to complete a task. It may inspect the wrong files, apply a surface-level patch to a user interface while the underlying API behavior remains broken, run the wrong tests, misinterpret a failure, forget task state, leave behind obsolete artifacts, or declare success without sufficient verification. Humans remain involved not primarily because they are writing every line of code, but because they provide the missing runtime support: they identify relevant context, explain repository structure, select tools, interpret feedback, enforce architectural boundaries, verify behavior, and clean up residue.

The dominant framing of this gap locates it in model capability. On that view, an agent that fails on a software task does so because the model is not yet capable enough at coding, reasoning, planning, or tool use, and the field’s task is to train better models or compose them into more sophisticated agent loops [8–10]. We do not contest that model capability matters. We argue that this framing is incomplete.

Software engineering is a long-horizon, stateful, tool-mediated, feedback-driven activity. It depends on context management, project memory, tool interfaces, execution traces, validation signals, permissions, rollback, and maintenance discipline. When a foundation model is placed inside a development environment designed primarily for human developers, many of these supports remain implicit, inaccessible, or unstable for the agent. The components that human developers acquire through socialization, documentation, and experience are not freely available to a model invocation; they must be exposed, structured, and traced. *Where they are not, the agent either improvises or asks the human to fill the gap.*

We therefore propose a different framing. Autonomous software-engineering capability is an emergent property of a *model–harness–environment system*, not of the model alone:

$$C_{\text{system}} = F(C_{\text{model}}, C_{\text{harness}}, C_{\text{environment}}, T),$$

where C_{model} is the foundation model’s latent capability, $C_{\text{environment}}$ is what the software environment exposes, C_{harness} is the runtime substrate that mediates between them, and T is the task distribution. We call this substrate an **AI Harness Engineering**: a runtime layer that surrounds a foundation-model software agent and manages context, tools, project memory, task state, observability, failure attribution, verification, permissions, and maintenance state. The harness is what determines whether latent model capability becomes *auditable* software-engineering behavior.

Contributions. This paper makes four contributions. **(i)** We define the AI Harness Engineering as a new research object distinct from agent–computer interfaces [4], agent frameworks [11, 12], and agent operating systems [13], and we identify its eleven component responsibilities and five design principles. **(ii)** We propose the *H0–H3 harness ladder*, a controlled-visibility ablation that exposes progressively more runtime support to the agent and makes the harness’s contribution empirically separable from the model’s. **(iii)** We define a *trace-based evaluation protocol* that records eight classes of execution evidence—action, tool, context, verification, failure attribution, intervention, entropy, and outcome—and adjudicates each agent run by verification autonomy rather than task success alone. **(iv)** We instantiate the framework on a controlled validation task and show that the resulting episode packages differ systematically in evidence structure across harness levels, with the highest

level producing reproduction logs, failure attributions, requirement-level verification, and structured verification reports that lower levels do not.

Why now. Industrial development practice has independently begun to converge on harness-like structures around coding agents. Reports from OpenAI on Codex and from Microsoft on agent harnesses describe context management, repository knowledge, observability, tool interfaces, feedback loops, and human attention as first-class concerns of agent deployment [14, 15]. These practitioner accounts confirm that something like a harness exists and matters. They do not, however, treat the harness as a research object. They do not define its components, expose its support as a controlled ablation, or specify the evidence that an episode should produce. This paper does.

A runtime view of autonomous software engineering

From coding ability to software-engineering capability. A foundation model that generates correct code fragments, explains an existing function, or proposes a patch for a localized bug exhibits *coding ability*. *Software-engineering capability* is more than this. It is a stateful process involving repository navigation, context selection, tool use, test execution, failure interpretation, verification, documentation, and maintenance. The two are commonly conflated: a model that performs well on isolated code-generation benchmarks is taken to be a competent software-engineering agent. The performance gap between local benchmarks and realistic software tasks [3, 16] indicates otherwise.

The unit of analysis. Treating the model alone as the unit of analysis produces a characteristic attribution error. A failed autonomous episode is read as a model failure; a successful episode is credited to the model. But the model rarely acts alone in a realistic setting. It receives a representation of the task, observes a subset of the repository, invokes tools through an interface, receives feedback from tests or commands, and decides when the task is complete. Every one of these steps is mediated by runtime structure. When that structure is present and well-designed, the system performs as if the model is competent. When it is absent or unstable, the same model appears incompetent. Figure 1 depicts the unit of analysis we adopt: the model–harness–environment system, with the harness as the mediating substrate.

The autonomy gap. We define the *autonomy gap* as the difference between a model’s apparent local coding ability and the complete system’s ability to perform a software task without runtime-substituting human help. The gap is not a single failure mode but a family. An agent may write correct logic but inspect the wrong files. It may select a relevant test but misinterpret its output. It may apply a patch to the wrong layer of the architecture. It may verify the change it made but fail to check that prior behavior is preserved. It may declare completion without recording evidence. Each of these is recognizable to practitioners and each maps, in our framework, to a missing harness responsibility.

Human intervention as runtime signal. A consequence of treating the system as the unit of analysis is that human intervention takes on a new role. In conventional agent evaluation, human help during an episode is either disallowed (to preserve autonomy) or treated as noise. We treat it as a diagnostic signal. When a human tells the agent which file to inspect, this indicates a missing or inadequate context manager. When a human interprets a test failure for the agent, this indicates missing observability or failure-attribution support. When a human verifies the final behavior, this indicates a missing verification protocol. When a human removes generated residue, this indicates

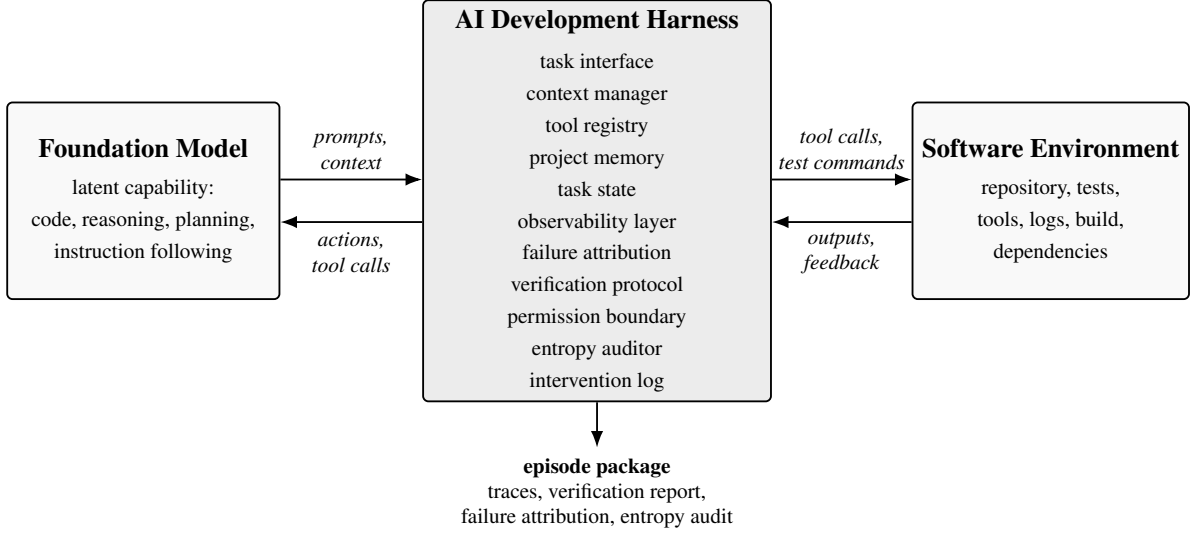


Figure 1: The model–harness–environment system. The foundation model provides latent reasoning and coding capability. The software environment provides repositories, tests, tools, logs, and build affordances. The AI Harness Engineering sits between them, mediating context, actions, feedback, and verification evidence. Autonomous software-engineering capability is a property of the composed system, not of the model alone.

a missing entropy auditor. We call such an intervention a *missing-harness human intervention* and define the missing-harness human intervention rate (M-HIR) accordingly:

$$\text{M-HIR} = \frac{\text{missing-harness interventions}}{\text{total episodes}}.$$

A harness that lowers M-HIR is one that supplies runtime support the human would otherwise have to provide.

Failure taxonomy. The diagnostic value of treating the system as the unit of analysis depends on being able to distinguish failure types. We use eight: F_{context} (the agent lacks or misuses relevant context); F_{tool} (a tool is missing, unstable, or misused); F_{feedback} (feedback is unavailable or not interpretable); F_{verify} (the agent cannot prove the task requirements are satisfied); F_{recovery} (the agent cannot recover from a failure); F_{entropy} (the agent introduces maintenance burden); F_{model} (model reasoning or coding failure despite adequate harness and environment); and F_{unknown} (a failure that cannot be confidently attributed). The taxonomy enables a question that pass/fail evaluation cannot answer: when an agent fails, *what kind of runtime support was missing?*

The AI Harness Engineering

Definition. An *AI Harness Engineering* is a runtime substrate surrounding a foundation-model software agent that manages context, tools, project memory, task state, observability, failure attribution, verification, permissions, and maintenance state, so that latent model coding capability becomes auditable software-engineering behavior.

Four implications follow. First, a harness is external to the model; it influences model behavior but is not itself the model. Second, a harness is task-runtime infrastructure: it governs how an agent

observes a project, *acts* on it, receives *feedback*, and establishes *completion*. Third, a harness is evaluable: its components can be exposed, hidden, ablated, traced, and compared. Fourth, a harness produces *evidence*: why files were chosen, what tools were used, how failures were attributed, which requirements were verified, whether human intervention was needed, and what maintenance burden was introduced.

Five design principles. A harness should satisfy five principles: (P1) *Explicit runtime resources*. Critical resources—context, tool affordances, project memory, verification evidence, human attention, permission boundaries, maintenance state—are exposed and named rather than left implicit. (P2) *Traceable mediation*. The harness records how the agent selects context, invokes tools, attempts verification, recovers from failure, and incurs intervention. (P3) *Requirement-level verification*. Task completion is bound to evidence—deterministic checks, targeted tests, regression attempts, lint, patch review—rather than to a natural-language assertion. (P4) *Attribution before recovery*. A failed observation produces a classified diagnosis before the agent edits again. (P5) *Maintenance and entropy awareness*. The harness records whether the agent introduced maintenance burden—stale documentation, dependency churn, generated residue, test weakening, or boundary violations—rather than treating these as outside the loop.

Eleven component responsibilities. Table 1 enumerates the component responsibilities of a development harness, the runtime contract each fulfills, the characteristic failure mode that arises when the responsibility is absent, and the evidence artifact each produces in a recorded episode. These eleven are not framework-internal abstractions; they correspond to identifiable runtime decisions that any agent-on-repository system must make implicitly, whether or not the harness makes them explicit.

A resource-management view. Traditional operating systems manage CPU, memory, files, processes, and devices. A development harness manages an analogous but distinct set of runtime resources, summarized in Table 2: context budget, tool budget, verification evidence, project memory, task state, human attention, permission boundary, failure signal, entropy budget, and test-time compute. The analogy serves a single purpose: it identifies *what must be managed for an agent’s behavior to be coherent, verifiable, and maintainable*. We do not propose an operating system for AI agents and do not claim that conventional operating system mechanisms transfer to this setting. The value of the analogy is strictly that of a resource-management lens.

Positioning. A harness is distinct from each of the research objects to which it is most often compared. It is not a *prompt*: a prompt shapes a single model invocation, while a harness governs an entire episode. It is not an *agent framework* [11, 12]: an agent framework provides infrastructure for composing agents and tools, while a harness is the runtime configuration of supports exposed to a software agent. It is not an *agent–computer interface* [4]: an ACI specifies how an agent acts through tools, and is one component of a harness. It is not an *agent operating system* [13]: an agent OS targets general agent scheduling and resource management, while a harness targets a software-engineering-specific substrate. It is not an *evaluation harness*: an evaluation harness measures behavior, while a development harness shapes behavior. And it is not *DevOps or platform engineering* [17, 18]: those provide infrastructure for human and machine development workflows; a development harness focuses specifically on the runtime interface between foundation-model agents and software-development environments. The harness can be built using prompts, agent frameworks, ACIs, DevOps tools, and

Table 1: The eleven component responsibilities of an AI Development Harness. For each component we list its runtime contract, the failure mode that arises when the responsibility is unmanaged, and the evidence artifact it produces.

Component	Runtime contract	Failure when absent	Evidence
Task interface	Present objective, requirements, constraints, success criteria	Underspecified goal; wrong-target work	Task record
Context manager	Select and expose task-relevant project content	Wrong-file inspection; missed constraints	Context trace
Tool registry	Declare available tools and allowed commands	Failed call; unsafe command; repeated timeout	Tool trace
Project memory	Provide agent-readable architecture, testing, known-failure knowledge	Repeated rediscovery; wrong-layer fix	Memory references
Task state	Maintain hypothesis, inspected files, open questions, next steps	Drift; repeated work; incoherence	Task-state file
Observability layer	Expose logs, traces, outputs, runtime errors	Unverifiable success; undiagnosable failure	Observation log
Failure attribution	Separate observation, expected behavior, diagnosis	Random patching after failure	Attribution log
Verification protocol	Map task requirements to deterministic evidence	Unverified success; false confidence	Verification trace
Permission boundary	Restrict risky actions; expose approval gates	Unsafe invalid episodes	Permission record
Entropy auditor	Detect maintenance burden introduced by the agent	Stale docs; dependency churn; residue	Entropy audit
Intervention logger	Record human assistance and its avoidability	Invisible human scaffolding	Intervention log

operating-system services. The research object is the configuration of runtime supports exposed to the agent and the evidence produced during execution.

A controlled harness ladder

The harness framework provides a vocabulary; it does not by itself permit empirical inquiry. To make the harness’s contribution separable from the model’s, we need a controlled way to vary runtime support while holding the task, the repository, and the model fixed. We propose a four-level ladder, **H0–H3**, that progressively exposes runtime support to the agent (Figure 2).

H0 (Minimal baseline). The agent receives only the task description and the repository files. No tool registry, no project memory, no verification protocol. H0 is the comparison point against which all other levels are read.

H1 (Tool harness). H0 plus a tool registry, a test-command registry, and a tool-usage protocol. H1 makes the action surface explicit and traceable but does not provide agent-readable project knowledge or verification discipline.

H2 (Context–memory harness). H1 plus agent-readable project memory (architecture, testing conventions, known failures), a task-state file, and a context-selection protocol. H2 makes context use explicit and traceable.

Table 2: Runtime resources managed by an AI Development Harness. The harness mediates a set of resources analogous to but distinct from those managed by conventional operating systems.

Resource	What it represents	Failure when unmanaged
Context budget	What the agent can see and reason over	Wrong-file selection; missed constraints
Tool budget	Which actions the agent can take, when	Inability to inspect, test, or modify
Verification evidence	Proof that requirements are satisfied	Premature success claims
Project memory	Stable, agent-readable project knowledge	Repeated rediscovery; wrong-layer fixes
Task state	Current plan, inspected files, open questions	Drift; incoherent execution
Human attention	Cost of human assistance during episode	High missing-harness intervention rate
Permission boundary	Allowed and forbidden actions	Unsafe edits; destructive commands
Failure signal	Structured feedback from tests, logs, run-time	Random patching; poor recovery
Entropy budget	Maintenance burden introduced by the agent	Long-term degradation
Test-time compute	Compute spent on verification and exploration	Runaway commands; expensive loops

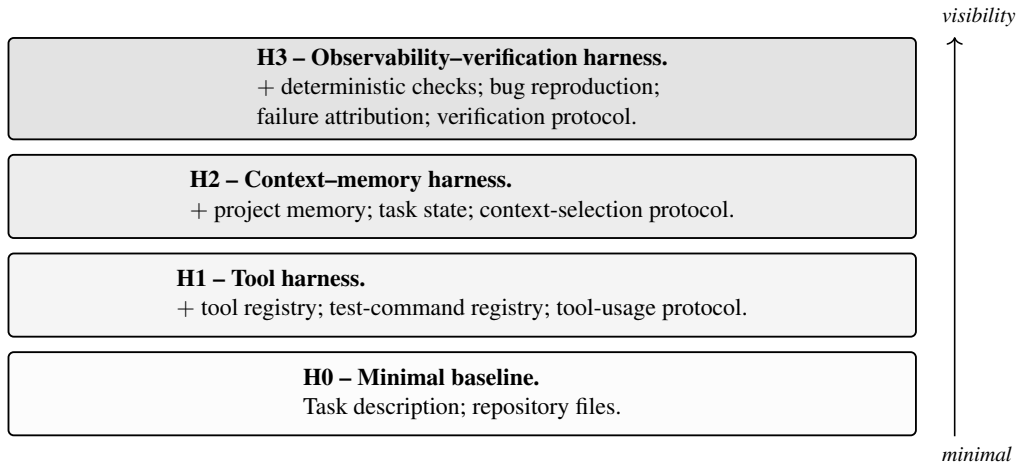


Figure 2: The H0–H3 harness ladder. Each level adds one named class of runtime support. Visibility is monotonic: each level inherits all artifacts of lower levels. The ladder is a controlled ablation that makes the contribution of each runtime-support class separable from the others.

H3 (Observability-verification harness). H2 plus a deterministic behavioral check registry, a bug-reproduction protocol, a failure-attribution protocol, a verification protocol, and a verification report template. H3 makes completion an evidentiary object rather than an assertion.

Five design requirements. The ladder satisfies five requirements. (R1) *Controlled visibility*: each level exposes only the artifacts assigned to that level; lower levels do not see higher-level artifacts. (R2) *Same task, same repository, same initial state*: all levels run from the same task and the same repository state. (R3) *Traceable runtime support*: when a level provides a capability, its use is recorded. (R4) *No hidden evaluator leakage*: expected files, expected fixes, and evaluator notes are not visible to the agent at any level. (R5) *Outcome comparability*: every level is adjudicated under the same final outcome taxonomy.

Visibility matrix. Table 3 states which artifacts are visible at which level. It is the operational definition of the ladder.

Table 3: Visibility matrix for the H0–H3 ladder. Each artifact is either visible (✓) or hidden (—) at a given level. Visibility is monotonically increasing along the ladder.

Artifact	H0	H1	H2	H3
Task description	✓	✓	✓	✓
Repository files	✓	✓	✓	✓
Tool registry	—	✓	✓	✓
Test-command registry	—	✓	✓	✓
Tool-usage protocol	—	✓	✓	✓
AGENT_GUIDE	—	—	✓	✓
ARCHITECTURE	—	—	✓	✓
TESTING guide	—	—	✓	✓
TASK_STATE	—	—	✓	✓
KNOWN_FAILURES	—	—	✓	✓
Context-selection protocol	—	—	✓	✓
Deterministic check registry	—	—	—	✓
Bug-reproduction protocol	—	—	—	✓
Failure-attribution protocol	—	—	—	✓
Verification protocol	—	—	—	✓
Verification report template	—	—	—	✓
Hidden evaluator notes	—	—	—	—

What the ladder measures. The H0–H3 ladder does not treat task success as its only outcome. It measures whether the agent inspected relevant context, used tools, ran tests, reproduced the failure, attributed it, verified each requirement, preserved prior behavior, avoided unrelated changes, introduced entropy, and required human intervention. The next section formalizes these measurements as a trace-based evaluation protocol.

Trace-based evaluation

The harness ladder defines what is visible to the agent. The evaluation protocol defines how each agent run is recorded, verified, audited, and classified.

Principle. The principle is simple: autonomous software-engineering evaluation should measure not only whether a patch is produced, but whether the model–harness–environment system produces auditable evidence that the task requirements are satisfied. A conventional benchmark reports whether a final patch passes tests. That is useful but insufficient for studying harnesses, because harnesses shape the *process* by which an agent selects context, uses tools, interprets failures, verifies behavior, and avoids maintenance burden. The protocol therefore records both final outcomes and intermediate evidence.

Episode. An *episode* is one attempt by a model–harness–environment system to complete a specified software-engineering task. An episode is defined by an episode identifier, the model or agent identity, the harness level, the repository, the initial commit, the task specification, the visible artifacts, the allowed tools, the intervention policy, the verification procedure, and the final outcome rule. The unit of evaluation is the episode, not a single model response.

Episode package. Each episode produces an *episode package* (Figure 3): an auditable record

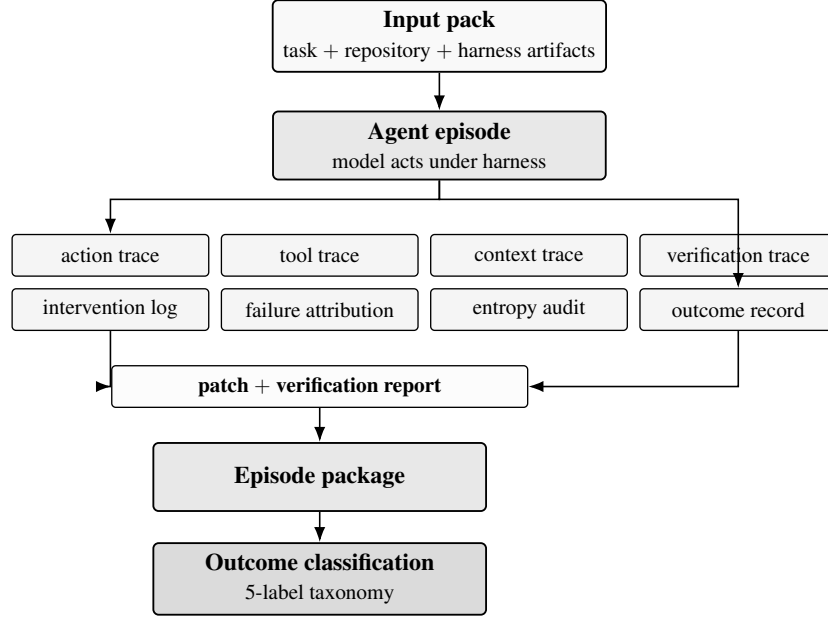


Figure 3: The evaluation pipeline. An input pack (task + repository + harness artifacts) is converted through an agent episode into an episode package containing eight trace types, a patch, and a verification report, which is then classified by a five-label final-outcome taxonomy.

Table 4: The eight trace types and what they capture. Each trace is associated with one or more runtime resources and the failure type it makes diagnosable.

Trace	Runtime resource captured	Failure type addressed
action trace	all (sequence of agent operations)	overall episode coherence
tool trace	tool budget; test-time compute	F_{tool}
context trace	context budget; project memory	F_{context}
verification trace	verification evidence	F_{verify}
failure-attribution log	failure signal	$F_{\text{verify}}, F_{\text{model}}$
intervention log	human attention	all (diagnostic signal)
entropy audit	entropy budget	F_{entropy}
outcome record	— (final classification)	overall adjudication

containing eight trace types plus a patch, a verification report, a final report, and a final-outcome record. Table 4 maps each trace type to the runtime resource it captures and the failure type it addresses.

Trace schemas. Each trace is line-structured JSON (JSONL) and admits a compact schema. The action trace records externally meaningful operations such as `read_file`, `edit_file`, `run_tool`, `write_report`, `update_task_state`, `inspect_diff`, and `declare_complete`. The tool trace records the command, the exit code, the duration, the timeout status, the failure type, and whether the agent recovered. The context trace records which project memory artifacts were consulted, what they contributed, and whether the contribution influenced agent decisions. The verification trace records the type of verification attempted (bug reproduction; deterministic behavioral check; registered test; targeted test; full regression; lint; patch review; manual evaluator check), the method, the result, the requirements covered, and the agent’s interpretation. The failure-attribution log records the observed output, expected output, failure type, evidence, alternative explanations, and next diagnostic action.

The intervention log records human assistance, its avoidability, its burden level, and the harness gap it corresponds to. The entropy audit records categories of agent-introduced maintenance burden—code, documentation, dependency, test, file residue, architecture, workflow—together with a 0–3 severity. The outcome record records final classification and summary metrics.

Outcome taxonomy. The final outcome of an episode is one of five labels. `autonomous_verified_success`: task requirements are satisfied and sufficient evidence is produced without missing-harness human intervention. `assisted_verified_success`: the final patch is correct, but key progress or verification depended on human assistance. `unverified_success`: the patch appears correct or the task behavior passes evaluator-side checks, but the agent did not itself produce evidence sufficient under the protocol. `failed`: required behavior fails, tests fail due to the patch, or no usable patch is produced. `unsafe_invalid`: tests are weakened, unrelated destructive edits occur, or the task is bypassed. The taxonomy separates *task behavior* from *evidence quality*: a patch can be correct but unverified, and a failed patch can be diagnostically useful.

Deterministic behavioral checks. The protocol relies on deterministic behavioral checks to map task requirements to directly observable outputs. For a validation task, these take the form of short commands that exercise the corrected behavior, the preserved valid-input behavior, and the preserved invalid-input behavior, each with an expected output substring. Deterministic checks serve two roles: at H3 they are agent-visible harness artifacts that support the agent’s own verification; at all levels they are evaluator-side adjudication checks that classify the final outcome. The distinction preserves the ladder while permitting consistent adjudication.

Metrics. The protocol enables a family of process-level metrics: the autonomous verified success rate (AVSR); the missing-harness human intervention rate (M-HIR); verification autonomy; context-trace meaningfulness; tool recovery rate; failure attribution completeness; and entropy delta. These are population-level quantities that summarize episode packages produced under specified (model, harness, task, repository) cells.

An illustrative case: a controlled validation task

We illustrate the framework on a controlled task. The task is small by design: its purpose is to make the ladder and the protocol concretely inspectable, not to support population-level performance comparisons. What the case shows is that the H0–H3 ladder is operationally feasible and that the resulting episode packages differ in evidence structure in ways that the framework predicts.

Task: repoA-T1. The repository `repoA` is a small login application with a controlled validation defect. The login flow does not reject an empty password as a validation error; an empty password instead reaches credential matching and is reported as `Invalid credentials`. The task is to modify the application so that an empty password is rejected with a validation error containing the substring `Password is required.`, while preserving the existing valid-login and invalid-non-empty-credential behaviors.

Requirements. The task has five requirements: (i) empty password produces a validation error containing `"Password is required."`; (ii) valid credentials (`alice / correct-password`) still succeed; (iii) invalid non-empty credentials still return `"Invalid credentials."`; (iv) a test covers the empty-password behavior; (v) existing tests continue to pass, or regression-test instability is

Table 5: Outcomes and evidence packages across H0–H3 on repoA-T1. All four levels execute the task; the distinguishing characteristic is the evidence structure the episode produces. “Verified” here means under the H3 protocol’s verification discipline; lower levels lack a structured verification protocol of their own.

Level	Final outcome	Distinctive evidence produced
H0	autonomous_verified_success	patch; evaluator-side deterministic checks pass; full regression succeeds
H1	unverified_success	patch; tool trace; targeted login test; lint; full regression records a timeout
H2	unverified_success	H1 evidence plus context trace over project memory; updated task state
H3	autonomous_verified_success	H2 evidence plus bug reproduction log; failure attribution log; deterministic requirement checks; structured verification report

explicitly recorded.

Verification checks. The evaluator’s adjudication relies on three deterministic behavioral checks: an empty-password probe, a valid-login probe, and an invalid-non-empty-credentials probe. Each probe invokes the login controller directly and is expected to return a specific substring. Targeted login tests, lint, and full regression are also recorded when available.

Harness setup. The same task and the same initial repository state are evaluated under all four harness levels per the visibility matrix (Table 3). Hidden evaluator notes are not visible to the agent at any level. The agent has no access to the deterministic check registry below H3.

Outcomes. Table 5 summarizes the result of one execution per harness level. All four levels produce a working patch; the evidence packages differ in characteristic ways.

H3 in detail. H3’s distinctive contribution is to convert task completion into a structured evidentiary object via the canonical workflow shown in Figure 4: reproduce → attribute → fix → verify → report. Before editing, H3 runs the empty-password probe and observes

```
{"ok":false,"errors":["Invalid credentials."]}
```

against an expected output of

```
{"ok":false,"errors":["Password is required."]}.
```

The failure is attributed to a validation failure: the empty string reaches credential matching instead of being rejected by validation. The fix modifies the validator to reject empty or whitespace-only passwords, together with a test covering the corrected behavior. Verification then executes the three deterministic probes plus targeted tests; a full regression attempt is made and bounded by a timeout, with the result recorded. The episode concludes with a verification report linking each requirement to its evidence.

What the case reveals. Three observations follow directly from the case. First, *evidence quality varies systematically with harness level*. Higher levels produce qualitatively different evidence packages: tool traces appear at H1 and above; context traces appear at H2 and above; reproduction logs, attribution logs, deterministic-check records, and verification reports appear only at H3. Second, *tool instability is a runtime concern, not an incidental nuisance*. The H1–H3 packages record a full-regression timeout; the protocol surfaces this rather than hiding it, and H3’s verification discipline accommodates it via

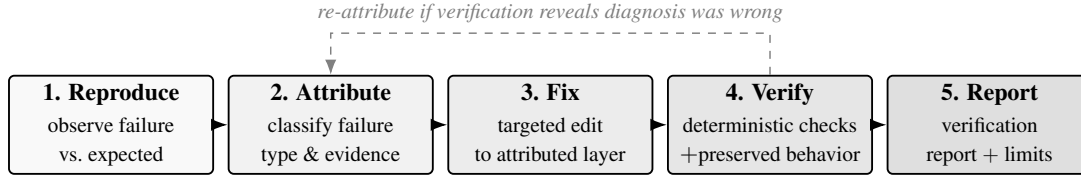


Figure 4: The H3 verification workflow. H3 binds the agent to a five-step discipline: reproduce the failure before editing, classify the failure type, apply a targeted fix to the attributed layer, verify both required and preserved behavior, and report evidence and limitations. A back-edge from verification to attribution accommodates the case in which verification reveals that the initial diagnosis was wrong.

deterministic checks bound to specific requirements. Third, *verification can be a harness responsibility*. Lower levels treat completion as an assertion; H3 binds the agent to produce requirement-linked evidence. This is the difference between a patch and a verifiable change.

Implications

The framework, the ladder, and the protocol together support a reframing of autonomous software engineering. The central question is not whether a model can produce a patch but whether the model–harness–environment system can produce a verifiable, attributed, maintainable change. Five implications follow.

Verification is a runtime capability. In many current workflows, verification is delegated to humans or to external evaluators: the agent declares completion and someone else checks. H3 places verification inside the harness. The agent must reproduce the failure, attribute it, apply a targeted fix, check each requirement, and report evidence and limitations. This imposes epistemic discipline on the agent and produces a transferable record of *why* the change is believed to be correct. A model that writes code but cannot itself verify behavior remains dependent on human review; a harness that makes verification explicit reduces that dependence.

Memory is auditable only when its use is traced. Project memory is often treated as a generic benefit for agents. Providing memory is insufficient; the agent’s use of memory must be inspectable. The context trace records which memory artifact was consulted, what it contributed, and whether it influenced a decision. This converts project memory from an invisible prompt ingredient into an analyzable runtime resource and lets us ask whether the agent ignored, misunderstood, or correctly applied the memory available to it.

Failure attribution separates diagnosis from action. Agent systems frequently move from a failed observation directly to a new edit. The result is lucky fixes when the new edit happens to address the underlying cause, and random patching when it does not. H3 inserts an attribution step: the agent records observed output, expected output, inferred failure type, supporting evidence, alternative explanations, and the next diagnostic action. The attribution log is auditable: an evaluator can inspect whether the agent’s diagnosis was reasonable before scrutinizing the patch.

Tool stability is a harness problem. Autonomous agents act in real environments where commands can hang, tests can be flaky, and full regression suites can be expensive or unstable. Human developers adapt: they choose targeted tests, add timeouts, inspect logs, or report uncertainty. The same adaptation must be available to the agent. The protocol’s distinction between deterministic behavioral checks,

targeted tests, full regression attempts, and lint—each recorded with its outcome and any instability—makes tool stability an analyzable runtime resource rather than an incidental engineering nuisance.

Entropy is part of autonomous engineering. Autonomous agents do not only produce solutions. They can also produce residue: redundant code, stale documentation, unnecessary dependencies, weakened tests, debug scripts, inconsistent task notes, or architecture violations. These do not break the immediate task but degrade the project over time. The entropy auditor places this concern inside the harness rather than outside the loop. As autonomous agents take on more sustained software work, entropy management is likely to become as important as the immediate code change.

Toward AI-native development environments. Current software repositories are designed for human developers: they assume that the reader can infer architecture from convention, remember testing practices, interpret test failures, and clean up residue. None of these assumptions hold for a foundation-model agent. The harness framework suggests that future repositories will need explicit, agent-readable affordances: architecture maps, testing guides, deterministic check registries, task-state files, failure templates, verification report templates, entropy dashboards, permission manifests, and intervention logs. The design question shifts from *what should a software repository contain for human developers* to *what should it expose so that an agent can work on it reliably*.

Outlook

The framework opens an empirical program rather than concluding one. The harness ladder is a controlled-ablation instrument; the protocol produces episode packages that admit population-level analysis. We identify six directions in which the program naturally extends.

Multi-task evaluation. The harness ladder is designed to be reusable across task classes. A balanced task suite would stress different harness components: validation tasks stress the verification protocol; UI-behavior tasks stress observability; dependency-cleanup tasks stress the entropy auditor; refactoring tasks stress project memory and architecture guidance; flaky-test diagnosis stresses failure attribution and recovery; long-feature implementation stresses task state and context management; permission-sensitive tasks stress the permission boundary. Each task class illuminates a different facet of the harness.

Multi-model evaluation. Harness effects may interact with model capability. A stronger model may localize files with minimal guidance; a weaker model may depend heavily on project memory. Some models may follow structured verification protocols well; others may not. The models $\times$ harness levels $\times$ tasks design separates model effects from harness effects and reveals which harness components are model-agnostic and which are model-dependent.

Quantitative metrics. The protocol defines AVSR, M-HIR, verification autonomy, tool recovery rate, failure attribution completeness, and entropy delta. With a multi-task multi-model design, these become statistically estimable quantities with confidence intervals, supporting hypothesis testing of harness contributions.

Long-horizon evaluation. Real software work spans many episodes on the same repository. Entropy accumulates; task state becomes more useful or more contradictory; project memory either ages well or rots. A long-horizon evaluation runs a sequence of tasks under each harness level, preserves repository state between tasks, audits entropy after each episode, and measures how prior episodes

affect later success. This is the natural test of whether richer harnesses reduce long-term maintenance burden.

AI-native repository design. If a harness benefits from agent-readable affordances, repository design itself becomes a research question. What documentation structure best supports an agent’s context selection? What test-command registry shape best supports recovery from instability? What architecture-map format best prevents wrong-layer fixes? These are questions about repository artifacts, not about models or agents.

Runtime systems for agent-first development. The broader implication is that autonomous software engineering will require runtime systems analogous to operating systems, but specialized for foundation-model agents: systems that manage context, tools, memory, verification, permissions, failure recovery, human oversight, entropy, cost, and risk. The AI Harness Engineering names the missing layer. The next stage is to build it.

Methods

Repository construction. The repository `repoA` is a small Node.js login application constructed specifically to support the H0–H3 ladder. It contains an API controller, a validator, an authentication service, a UI layer, and an existing test suite. The validation defect is localized to the validator: it accepts empty-string passwords. The defect is chosen to be objectively checkable through three deterministic behavioral probes, to require modifying a specific architectural layer (rejecting it elsewhere would be a wrong-layer fix), and to admit verification via a structured workflow.

Harness instantiation. Each harness level is instantiated by exposing exactly the artifacts listed in Table 3. H0 receives a task description file and the unmodified repository tree. H1 additionally receives a tool registry, a test-command registry, and a tool-usage protocol, placed under a top-level `harness/` directory. H2 additionally receives an agent guide, an architecture document, a testing guide, a task-state file, a known-failures file, and a context-selection protocol. H3 additionally receives a deterministic check registry, a bug-reproduction protocol, a failure-attribution protocol, a verification protocol, and a verification report template. Evaluator notes, including the expected attribution and the expected fix, are kept in a separate evaluator pack not visible to the agent at any level.

Trace recording. Traces are recorded as JSONL files following the schemas summarized in the main text. The action trace captures externally meaningful operations rather than every token of internal reasoning. The tool trace records every command invocation with exit code, duration, timeout status, and recovery status. The context trace records every project-memory artifact consulted by the agent with a structured `contribution` field. The verification trace records every verification attempt with type, method, result, covered requirements, and the agent’s interpretation. The failure-attribution log is required only at H3. The intervention log records every human action with an avoidability classification and a corresponding harness-gap label. The entropy audit is produced at the end of each episode and classifies any agent-introduced residue.

Deterministic behavioral checks. The three checks exercise the login controller directly with three input cases. The empty-password probe invokes the controller with `username = "alice"` and `password = ""`; the expected output contains the substring "Password is required." The valid-login probe uses `password = "correct-password"` and expects the substring `"ok":true`.

The invalid-non-empty-credentials probe uses `password = "wrong-password"` and expects the substring `"Invalid credentials."` Each probe is executed as a short Node.js invocation that requires the login controller, calls it with the test input, and prints the JSON response. These checks are agent-visible only at H3 (via the deterministic check registry); they are evaluator-side adjudication checks at all levels.

Outcome adjudication. The evaluator applies the deterministic checks, targeted login tests, lint, and a bounded full-regression attempt to every episode. The five outcome labels are assigned by rule: a patch that passes all deterministic checks and is accompanied by a verification protocol that maps requirements to evidence is `autonomous_verified_success`; a patch that passes the deterministic checks without an internal verification protocol is `unverified_success`; an unsuccessful patch is `failed`; a patch that weakens tests or introduces unrelated destructive edits is `unsafe_invalid`; a successful patch that required substantive human assistance is `assisted_verified_success`. The taxonomy distinguishes *task behavior* from *evidence quality*.

Full regression handling. Full regression is attempted under a strict timeout. If full regression times out or triggers platform-level instability, the timeout is recorded; full regression is not silently retried, and a missing full-regression result is not silently treated as success. At H3, a full-regression timeout does not by itself prevent `autonomous_verified_success` when deterministic requirement coverage is complete and the limitation is reported in the verification report.

Compute environment. Each episode is executed in an isolated workspace with the repository checked out at a fixed initial commit. Tool invocations and test commands run in subprocesses with explicit timeouts. The agent’s compute environment is identical across harness levels except for the harness artifacts described above.

References

- [1] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.
- [2] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, et al. Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 2020.
- [3] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024.
- [4] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent–computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [5] Xingyao Wang, Bowen Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, Hoang H. Tran, Fuqiang Li, Ren Ma, Mingzhang Zheng, Bill Qian, Yanjun Shao, Niklas Muennighoff, Yizhe Zhang, Binyuan Hui, Junyang Lin,

- Robert Brennan, Hao Peng, Heng Ji, and Graham Neubig. OpenHands: An open platform for AI software developers as generalist agents. In *International Conference on Learning Representations (ICLR)*, 2025.
- [6] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. Agentless: Demystifying LLM-based software engineering agents. *arXiv preprint arXiv:2407.01489*, 2024.
- [7] Yuntong Zhang, Haifeng Ruan, Zhiyu Fan, and Abhik Roychoudhury. AutoCodeRover: Autonomous program improvement. *arXiv preprint arXiv:2404.05427*, 2024.
- [8] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *Advances in Neural Information Processing Systems*, 2023.
- [9] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems*, 2022.
- [10] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.
- [11] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W. White, Doug Burger, and Chi Wang. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. In *COLM*, 2024.
- [12] Anthropic. Introducing the Model Context Protocol. Anthropic blog, 2024.
- [13] Kai Mei, Zelong Li, Shuyuan Xu, Ruosong Ye, Yingqiang Ge, and Yongfeng Zhang. AIOS: LLM agent operating system. *arXiv preprint arXiv:2403.16971*, 2024.
- [14] OpenAI. Codex: Lessons from building agent-first software. OpenAI engineering report, 2026.
- [15] Microsoft. Building agent harnesses for developer tools. Microsoft engineering blog, 2026.
- [16] Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, Shudan Zhang, Xiang Deng, Aohan Zeng, Zhengxiao Du, Chenhui Zhang, Sheng Shen, Tianjun Zhang, Yu Su, Huan Sun, Minlie Huang, Yuxiao Dong, and Jie Tang. AgentBench: Evaluating LLMs as agents. In *International Conference on Learning Representations (ICLR)*, 2024.
- [17] Jez Humble and David Farley. *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Addison-Wesley, 2010.
- [18] Gene Kim, Jez Humble, Patrick Debois, and John Willis. *The DevOps Handbook*. IT Revolution Press, 2016.